TASK 6 – DESIGN THE INHERITANCE HIERARCHY

Campus Smart-Parking System

1. Introduction

Inheritance is an important object-oriented programming concept that allows a class to acquire the properties and methods of another class. It promotes code reusability, reduces duplication and supports extensibility.

In the Campus Smart-Parking System, vehicles have common characteristics such as registration number and parking fee calculation. However, different vehicle categories have different parking rates and additional capabilities. Therefore, an inheritance hierarchy is designed using a common superclass named `Vehicle` and specialized subclasses.

2. Superclass and Subclasses

The proposed vehicle inheritance hierarchy consists of the following classes:

1. Vehicle – Superclass

- Acts as the common superclass for all vehicle categories.
- Stores common attributes such as registration number.
- Provides common methods such as `getRegNo()`.
- Defines an abstract method `ratePerHour()` that must be implemented by subclasses.
- Cannot be instantiated directly because it represents a general vehicle.

2. TwoWheeler – Subclass

- Inherits the common properties and methods of Vehicle.
- Uses a parking rate of Rs. 10 per hour.
- Does not require additional vehicle-specific attributes.

3. Car – Subclass

- Inherits the common properties and methods of Vehicle.
- Uses a parking rate of Rs. 30 per hour.
- Implements its own version of the hourly rate method.

4. ElectricCar – Subclass

- Inherits the properties and behaviour of Car.
- Represents an electric vehicle.
- Obtains the car parking rate through inheritance.
- Adds the additional capability of charging.
- Can implement the Chargeable interface to provide a separate charging fee.

Type of Inheritance Used

The design uses both hierarchical and multilevel inheritance.

- **Hierarchical inheritance:** TwoWheeler and Car inherit from the same superclass Vehicle.
- **Multilevel inheritance:** ElectricCar inherits from Car, which inherits from Vehicle.

This structure avoids repeating common vehicle attributes and provides a clear way to introduce specialized vehicle categories.

3. Inherited and Specialized Members

Class	Inherited Members	Additional Members / Behaviour
Vehicle	Not applicable	Registration number, getRegNo(), abstract ratePerHour()
TwoWheeler	Vehicle properties and methods	ratePerHour() returns Rs. 10
Car	Vehicle properties and methods	ratePerHour() returns Rs. 30
ElectricCar	Vehicle and Car properties and methods	Charging capability through Chargeable interface

The registration number is maintained in the superclass, avoiding duplicate declarations in every subclass. Each vehicle category provides its specific hourly rate.

ElectricCar inherits the car parking rate while extending the functionality through its charging capability.

4. Use of the super Keyword

The `super` keyword refers to the immediate parent class. It is used to access parent-class constructors and methods.

A. Using super for Constructors

The subclass constructor calls the superclass constructor to initialize the inherited registration number.

Example:

```
class Vehicle {
    private String regNo;

    Vehicle(String regNo) {
        this.regNo = regNo;
    }
}

class Car extends Vehicle {
    Car(String regNo) {
        super(regNo);
    }
}
```

Here, `super(regNo)` invokes the Vehicle constructor and initializes the registration number.

B. Using super for Methods

A subclass can use `super.methodName()` to invoke an inherited method implementation.

Example:

```
class Car extends Vehicle {
    Car(String regNo) {
        super(regNo);
    }

    void displayDetails() {
        System.out.println(getRegNo());
    }
}
```

The inherited getter provides access to the registration number without directly accessing the private field.

C. Using super for Fields

The `super` keyword can access an accessible field declared in the parent class when a subclass field has the same name. However, in this design, the registration number is private and is accessed through its public getter to maintain encapsulation.

5. Constructor Execution Trace for ElectricCar

When an ElectricCar object is created, constructors execute from the topmost superclass to the most specialized subclass.

Consider the statement:

```
ElectricCar ev = new ElectricCar("TN03C3");
```

The inheritance hierarchy is: ElectricCar → Car → Vehicle

Constructor execution order:

Step	Constructor Executed	Operation
1	Vehicle(String regNo)	Initializes the registration number as TN03C3
2	Car(String regNo)	Executes after calling super(regNo)
3	ElectricCar(String regNo)	Executes after calling the Car constructor

Execution Flow:

```
new ElectricCar("TN03C3")
          |
          v
ElectricCar constructor
          |
          v
super(regNo)
          |
          v
Car constructor
          |
          v
super(regNo)
          |
          v
Vehicle constructor
          |
          v
Registration number initialized
          |
          v
Car constructor completes
          |
          v
ElectricCar constructor completes
```

Result: The Vehicle constructor executes first, followed by Car and finally ElectricCar. This ensures that the inherited state is initialized before the subclass initialization is completed.

6. User Inheritance Hierarchy

The system also involves three types of users: students, staff and visitors.

A separate user hierarchy can be designed as follows:

- **ParkingUser:** Common superclass containing user identification and common user information.
- **Student:** Inherits ParkingUser and may possess a valid monthly parking pass.
- **Staff:** Inherits ParkingUser and may possess a valid monthly parking pass.
- **Visitor:** Inherits ParkingUser and normally pays the applicable parking fee.

This hierarchy is kept separate from the vehicle hierarchy because a user and a vehicle represent different entities. A user owns or operates a vehicle, but a user is not a type of vehicle.

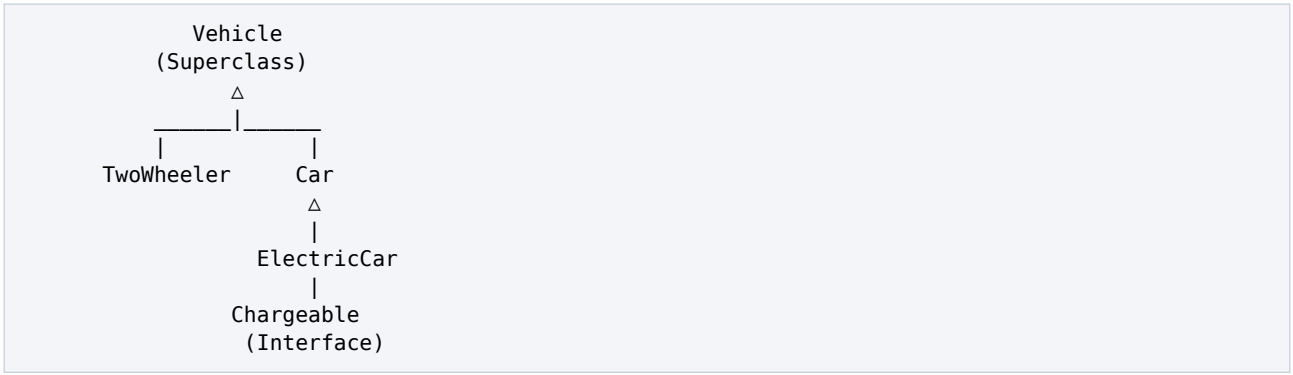
The separate hierarchy also allows the parking eligibility and pass-related rules to be extended independently.

7. UML Class Diagram

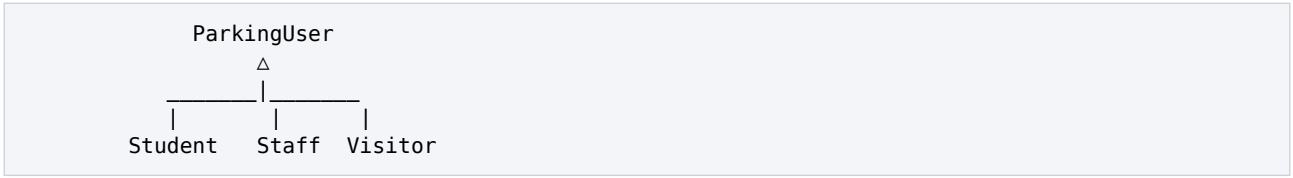
The UML diagram represents the inheritance relationships between the vehicle classes and the optional user hierarchy.

The hollow triangle arrow represents inheritance and points towards the superclass.

Vehicle Hierarchy:



User Hierarchy:



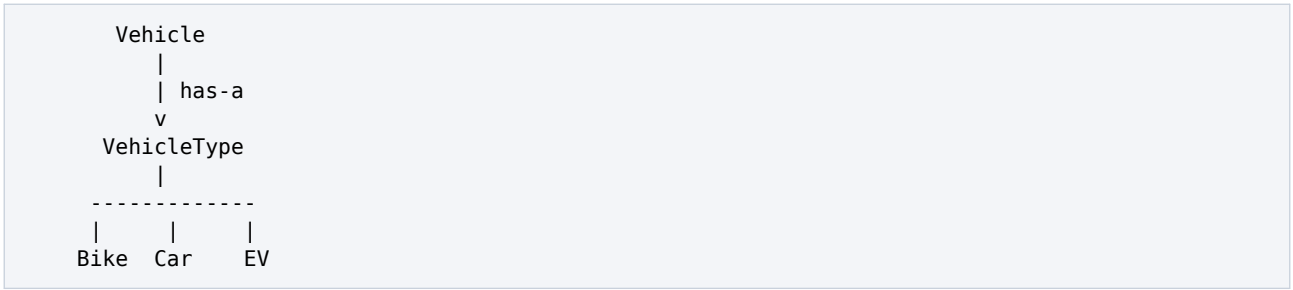
The vehicle and user hierarchies are independent. ElectricCar extends Car and can implement Chargeable to support the additional EV charging functionality.

8. Alternative Concept – Composition Instead of Inheritance

An alternative approach is composition, where a single Vehicle class contains a VehicleType object that stores the vehicle category and its hourly rate.

Instead of creating separate subclasses such as TwoWheeler, Car and ElectricCar, the Vehicle object would contain a reference to a VehicleType object.

Proposed Composition Structure:



For example, a Vehicle object can contain:

- Registration number
- VehicleType reference

The VehicleType object can store:

- Vehicle category
- Hourly parking rate
- Applicable pricing information

A fee calculation method would obtain the rate from the associated VehicleType object and calculate the parking fee.

Comparison Between Inheritance and Composition

Criteria	Inheritance	Composition
Relationship	Car is-a Vehicle	Vehicle has-a VehicleType
Design	Separate subclasses for vehicle categories	One Vehicle class with a VehicleType object
Adding a new vehicle type	Requires creating a new subclass	Requires creating a new VehicleType configuration or implementation
Code reusability	Achieved through superclass inheritance	Achieved through delegation to the VehicleType object
Fee calculation	Uses an overridden ratePerHour() method	Uses the rate stored or provided by VehicleType
Extensibility	Suitable for stable vehicle categories	More flexible when vehicle categories and pricing rules change frequently

Time Complexity Analysis

Assume that there are n parked vehicles and k vehicle types.

Inheritance Approach:

- Retrieving the hourly rate through the overridden `ratePerHour()` method takes O(1) time.
- Calculating the fee for a vehicle takes O(1) time.
- Dynamic method dispatch selects the appropriate subclass implementation without manually searching through all vehicle types.

Composition Approach:

- Accessing the associated VehicleType object takes O(1) time.
- Retrieving its rate and calculating the fee also takes O(1) time.
- Therefore, fee calculation remains O(1), provided the rate is directly available from the VehicleType object.

Space Complexity Analysis

Inheritance Approach:

Each parked vehicle requires one vehicle object containing its common fields and any additional subclass-specific fields. For n vehicles, the total storage is O(n), excluding shared class metadata.

Composition Approach:

Each vehicle requires a Vehicle object containing a reference to its VehicleType object. If vehicle types are shared among multiple vehicles, the type objects can be reused, reducing duplication. The total storage is O(n + k) when there are n vehicles and k shared type objects.

Effect of Adding a New Vehicle Type

In inheritance, introducing a new category requires creating another subclass that extends Vehicle and implements its specific behaviour. Existing subclasses generally remain unchanged.

In composition, a new vehicle category can be introduced by providing a new VehicleType configuration or implementation. The Vehicle class can remain unchanged if it delegates pricing behaviour to the VehicleType object.

Composition is more flexible when vehicle categories and pricing rules are expected to change frequently.

9. Conclusion

The inheritance-based design is suitable for the Campus Smart-Parking System because all vehicles share common properties but have different specialized behaviours. The Vehicle superclass provides common attributes and an abstract method, while TwoWheeler, Car and ElectricCar implement or inherit the required functionality.

Hierarchical and multilevel inheritance improve code reuse and maintainability. The `super` keyword ensures correct constructor chaining and supports access to inherited behaviour. The constructor trace confirms that superclass constructors execute before subclass constructors.

Although composition provides greater flexibility when vehicle types and pricing rules frequently change, inheritance offers a clear and understandable model for the current vehicle categories. Therefore, the inheritance hierarchy is retained for the original design, while composition remains a suitable alternative for future extensions.